

Relatorio de Arquitetura - Fase 2

Desenvolvimento de Sistemas Back-End - Fase 2

Aluno: Andreas Berwaldt

Data: Junho de 2026

1. Descricao do Projeto

O sistema implementa o backend de controle de planos de uma operadora de internet. A Fase 2 finaliza a arquitetura proposta na especificacao, integrando o servico principal a dois microservicos:

- **ServicoGestao**: gerencia clientes, planos e assinaturas.
- **ServicoFaturamento**: registra pagamentos em banco proprio e publica eventos.
- **ServicoPlanosAtivos**: responde rapidamente se uma assinatura esta ativa, usando cache.
- **api-gateway**: fornece uma entrada HTTP unica para as chamadas de teste.

A comunicacao assincrona usa RabbitMQ. O ServicoFaturamento publica evento de pagamento no exchange **pagamentos**; o ServicoGestao observa esse evento para atualizar **dataUltimoPagamento**, e o ServicoPlanosAtivos observa o mesmo evento para invalidar o cache da assinatura.

2. Arquitetura

A arquitetura da entrega segue o diagrama de componentes proposto na especificacao da disciplina: API Gateway como entrada HTTP, ServicoGestao como servico principal, ServicoFaturamento como microservico de pagamentos, ServicoPlanosAtivos como microservico de consulta rapida, dois bancos de dados independentes, cache e message broker.

Fluxo principal:

1. O cliente chama **POST /registrarpagamento** no API Gateway.
2. O Gateway encaminha a chamada ao ServicoFaturamento.
3. O ServicoFaturamento grava o pagamento em seu PostgreSQL.
4. O ServicoFaturamento publica evento no RabbitMQ.
5. O ServicoGestao consome o evento e atualiza **dataUltimoPagamento**.
6. O ServicoPlanosAtivos consome o evento e remove a assinatura do cache.
7. Na proxima consulta, o ServicoPlanosAtivos consulta o ServicoGestao em caso de cache miss.

3. Servicos

ServicoGestao

Mantem os dados de **Cliente**, **Plano** e **Assinatura** em PostgreSQL proprio. Continua seguindo Clean Architecture:

```
src/  
├─ domain/  
├─ application/
```

```
|— infrastructure/  
|— presentation/
```

Na Fase 2 foram adicionados:

- caso de uso para atualizar `dataUltimoPagamento` a partir de pagamento;
- caso de uso para verificar se uma assinatura esta ativa por codigo;
- endpoint interno `GET /gestao/assinatura/:codass/status`;
- consumer RabbitMQ para eventos de pagamento.

ServicoFaturamento

Possui banco PostgreSQL proprio com entidade `Pagamento`. O endpoint `POST /registrarpagamento` recebe `dia`, `mes`, `ano`, `codAss` e `valorPago`, persiste o pagamento e publica evento no RabbitMQ.

O nucleo foi construido com TDD:

- registrar pagamento valido;
- publicar evento apos persistir;
- rejeitar valor pago menor ou igual a zero;
- controller convertendo `dia/mes/ano` para `Date`;
- repositorio Prisma;
- publisher RabbitMQ.

ServicoPlanosAtivos

Responde `GET /planosativos/:codass` com booleano. Usa cache em memoria para reduzir consultas ao `ServicoGestao`.

Comportamento:

- cache hit: retorna o valor armazenado;
- cache miss: consulta `GET /gestao/assinatura/:codass/status` no `ServicoGestao` e armazena o resultado;
- evento de pagamento: invalida a entrada da assinatura paga.

API Gateway

Roda na porta `3000` e encaminha as rotas externas para os servicos internos:

- `/gestao/*` para o `ServicoGestao`;
- `/registrarpagamento` para o `ServicoFaturamento`;
- `/planosativos/:codass` para o `ServicoPlanosAtivos`.

4. Regra de Negocio

A especificacao define que o servico de internet requer pagamento a cada 30 dias, sem tolerancia de atraso. Por isso, a regra canonica implementada e:

```
get ativa(): boolean {
  const hoje = new Date();
  const diffMs = hoje.getTime() - this.dataUltimoPagamento.getTime();
  const diffDias = diffMs / (1000 * 60 * 60 * 24);
  return diffDias < 30;
}
```

O campo `fimFidelidade` permanece na entidade para representar fidelidade/promocionalidade, mas não determina se a assinatura está ativa.

5. Bancos e Broker

Componente	Tecnologia	Porta
Banco Gestao	PostgreSQL 16	5433
Banco Faturamento	PostgreSQL 16	5434
Broker	RabbitMQ	5672
Painel RabbitMQ	RabbitMQ Management	15672

O `docker-compose.yml` da raiz sobe os dois bancos e o RabbitMQ.

6. Testes e Validacao

Testes automatizados validados:

```
servico-gestao:      27 testes
servico-faturamento: 8 testes
servico-planos-ativos: 9 testes
api-gateway:         5 testes
```

Todos os builds TypeScript/NestJS também foram executados com sucesso.

Teste integrado validado via gateway:

```
GET /gestao/clientes      -> retornou 10 clientes
GET /planosativos/1       -> retornou true
POST /registrarpagamento -> registrou pagamento
GET /planosativos/1       -> retornou true apos evento/cache invalidado
```

Resposta observada no registro de pagamento:

```
{
  "codigo": 1,
```

```
"codAss": 1,  
"valorPago": 99.9,  
"dataPagamento": "2026-06-14T00:00:00.000Z"  
}
```

7. Desafios Encontrados

Desafio	Solucao
Integrar tres servicos mantendo responsabilidades separadas	API Gateway para entrada HTTP e RabbitMQ para comunicacao assincrona
Evitar duplicar regra de assinatura ativa no ServicoPlanosAtivos	O cache miss consulta o ServicoGestao, que continua dono da regra
Prisma 7 exige configuracao diferente das versoes anteriores	Uso de <code>prisma.config.ts</code> e <code>@prisma/adapters</code>
Injecao de dependencias com contratos abstratos no NestJS	Uso de <code>abstract class</code> como token e <code>@Inject(...)</code> quando necessario
Builds gerando <code>.js</code> dentro de <code>src</code> nos servicos novos	Configuracao de <code>rootDir</code> , <code>outDir</code> e <code>include</code> no <code>tsconfig.json</code>
Testar comunicacao assincrona sem depender de broker nos testes unitarios	Portas abstratas e adaptadores testados com fakes/mocks

8. Conclusao

A Fase 2 concluiu a arquitetura proposta no enunciado com os tres servicos solicitados, infraestrutura de broker, bancos independentes e cache para consulta rapida de assinaturas ativas. A abordagem incremental com TDD ajudou a manter as regras testaveis antes da integracao com NestJS, Prisma e RabbitMQ.

O sistema pode ser executado localmente com Docker Compose para a infraestrutura e quatro processos Node/NestJS para os servicos. A collection Postman foi atualizada para usar o API Gateway na porta 3000, conforme a estrutura final da entrega.

9. Referencias

- Documentacao oficial do NestJS: <https://docs.nestjs.com/>
- Documentacao oficial do Prisma: <https://www.prisma.io/docs>
- RabbitMQ Tutorials: <https://www.rabbitmq.com/tutorials>
- Clean Architecture - Robert C. Martin: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>